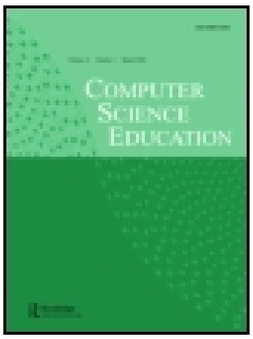

Computer Science Education

ISSN: 0899-3408 (Print) 1744-5175 (Online) Journal homepage: www.tandfonline.com/journals/ncse20

How Students Attempt to Reduce Abstraction in the Learning of Mathematics and in the Learning of Computer Science

Orit Hazzan

To cite this article: Orit Hazzan (2003) How Students Attempt to Reduce Abstraction in the Learning of Mathematics and in the Learning of Computer Science, Computer Science Education, 13:2, 95-122, DOI: 10.1076/csed.13.2.95.14202

To link to this article: https://doi.org/10.1076/csed.13.2.95.14202

Computer Science Education
2003, Vol. 13, No. 2, pp. 95–122

0899-3408/03/1302-095$16.00

How Students Attempt to Reduce Abstraction in the Learning of Mathematics and in the Learning of Computer Science

Orit Hazzan
Department of Education in Technology and Science, Technion – Israel Institute of Technology, Haifa, Israel

ABSTRACT

This article focuses on abstraction and ways in which students cope with abstraction. The article has two goals: first, it illustrates how the theme of reducing abstraction (Hazzan, 1999) is useful for analyzing students' thinking about abstract concepts in mathematics and in computer science; second, it demonstrates how theories based on mathematics education research can be applied to analyzing students' understanding of computer science concepts. The main section of the article analyzes the understanding of concepts from four fields – abstract algebra, computability, data structures and differential equations – through the lens of reducing abstraction. The analysis shows that a wide range of cognitive phenomena can be explained by one theoretical framework.

INTRODUCTION

This article focuses on abstraction and ways in which undergraduate students cope with abstraction. The article has two goals: first, it shows how the theme of reducing abstraction (Hazzan, 1999) is useful for analyzing students' thinking about abstract concepts in mathematics and in computer science; second, it illustrates how theories based on mathematics education research can be applied to analyzing students' understanding of computer science concepts. As the mathematics education discipline is relatively mature, with its roots extending from the close of the 19th century, mathematics educators have been studying learning theories, teaching approaches, and other

Address correspondence to: Orit Hazzan, Department of Education in Technology and Science, Technion – Israel Institute of Technology, Haifa, Israel. E-mail: oritha@techunix.technion.ac.il

cognitive and social issues with respect to K-12 and higher education. As it turns out some of these theories may be useful for gaining insights into how students learn and understand computing science topics, as well as for improving how we teach these topics (Almstrum, Ginat, Hazzan, & Morley, 2002). Thus, the attempt made in this article, of taking the theme of reducing abstraction developed in mathematics education and applying it to computer science education, may be a step towards mutual contribution of the two educational research fields.

The article starts with theoretical background which addresses two topics. First, the theoretical framework of reducing abstraction is outlined. Second, the theoretical background examines the nature of four fields – abstract algebra, computability, data structures and differential equations – the understanding of concepts from these fields being examined through the lens of reducing abstraction. The theoretical background is put to use in Coping With Abstraction section where students' ways of reducing abstraction in the context of mathematics and in the context of computer science are analyzed. This analysis illustrates that a wide range of cognitive phenomena can be explained by one theoretical framework.

THEORETICAL BACKGROUND

The theoretical framework of *reducing abstraction*[1] (Hazzan, 1999) describes undergraduate students' mental strategies for coping with abstraction. The framework was developed during a study whose aim was to explain students' perceptions of abstract algebra concepts. As it turns out, and as is widely illustrated in Coping With Abstraction section, the theoretical framework can be used for analyzing students' understanding of concepts in other fields as well.

From the perspective of reducing abstraction, most responses and conceptions on the part of students can be attributed to their tendency to work on a lower level of abstraction than the one on which concepts are introduced in class. The term 'reducing abstraction' should *not* be conceived as a mental process which necessarily results in misconceptions or mistakes. Rather, the mental process of reducing abstraction level indicates that students find ways to cope with new concepts they learn. The students make these

[1]The full title of the framework is *reduction of the level of abstraction*.

concepts mentally accessible so that they (the students) will be able to think with these concepts and handle them cognitively.

The theme of reducing abstraction is based on the following three interpretations of *levels of abstraction*:

i. abstraction level as the quality of the relationships between the object of thought and the thinking person (Wilensky, 1991);
ii. abstraction level as reflection of the process-object duality (Dubinsky, 1991; Sfard, 1991);
iii. abstraction level as the degree of complexity of the concept of thought.

In Coping With Abstraction section students' ways of reducing abstraction are analyzed. In the continuation of this section, the four subjects, whose understanding through the lens of reducing abstraction is analyzed in the next section, are described. The description does not consist of concept definitions. Rather, the short description of each field focuses mainly on the kind of problems and concepts that it deals with, and on the way it contributes to students' undergraduate education. Concept definitions that are mentioned in the article from each field are presented in the Appendix.

Abstract Algebra

An introductory course in abstract algebra deals with mathematical structures such as groups and rings, and with relationships among these structures such as homomorphism. It is usually the first undergraduate mathematical course in which students ''must go beyond learning 'imitative behavior patterns' for mimicking the solution of a large number of variations on a small number of themes (problems).'' (Dubinsky, Dautermann, Leron, & Zazkis, 1994, p. 268). Indeed, it is in the abstract algebra course that students are asked, for the first time, to deal with concepts which are introduced abstractly. That is, concepts are defined and presented by their properties and by an examination of ''what facts can be determined just from [the properties] alone.'' (Dubinsky & Leron, 1994, p. 42).

The importance of learning abstract algebra is widely acknowledged. Gallian (1990) says that ''abstract algebra is important in the education of a mathematically trained person. The terminology and methodology of algebra are used ever more widely in computer science, physics, chemistry, and data communications, and of course, algebra still has a central role in advanced mathematics itself.'' (p. 11). However, many instructors report difficulties on the part of the students in understanding ideas that they (the instructors) try to

communicate. Thus, educators try to find ways to help students understand abstract algebra concepts and look for ways, *relevant for the students*, to introduce these concepts. For example, in the introduction to his book *Abstract Algebra*, Herstein (1986) argues that since "[t]here is little purpose served in studying some abstract algebra object without seeing some nontrivial consequences of the study, [we present in the book] interesting, applicable, and significant results in each of the systems we have chosen to discuss." (p. 8). As can be observed, on the one hand the abstract nature of the concepts under discussion is emphasized; on the other hand, the relevance of these abstract structures to real life application is highlighted.

Computability

A typical computability course deals with assessing how difficult it is to solve computational problems. With respect to a given problem we ask questions such as: Is it *possible* to design an algorithm which solves the problem? Is it possible to design an *efficient algorithm* which solves the problem? Since most of the computer science students spend at least part of their professional life in software development, it is easy to observe the importance of computability to their professional education: In software development it is crucial to be able to examine whether or not a given problem is solvable. In cases where the problem is solvable, one must then go on examining whether the solution can be computed with enough efficiency that a computer-based solution can be used in practice. (Hopcroft, Motwani, & Ullman, 2001, p. 361).

In this paper only one kind of problems of computability theory is examined. The discussion is limited to questions in which one has to determine whether a given language is recursive or recursively enumerable (see Appendix). For determining whether a given language belongs to R (that is, it is recursive) or to RE (i.e., it is recursively enumerable), one has to understand the structure and meaning of the language. The better one understands the nature of a given language, the better one may determine to what category the language belongs. Then, based on this analysis, one has to choose a method for proving one's claim.

The objects with which students work in that part of the course that is addressed in this paper, are languages (which are sets of words described by a common property) and sets of languages. In these two cases the objects of thought are complex. For example, the Halting Problem, HP$=\{(\langle M\rangle, x)$: M halts on $x\}$, is a set of pairs – a coding of a Turing machine M and an input to

the machine – that share the property that M halts on x. Not only are the objects complex, they are addressed abstractly through their properties.

Data Structures

'Data structures' is a central subject in computer science. It deals with data, data organizations, and operations on these organizations. In other words, data structures refer not only to the data themselves but also to operations that are performed on the data.

Abstraction is a fundamental concept in the learning and in the use of data structures, as well as in the discipline's perspective at the topic. For example, Nell and Teague (2000) declare the following in the introduction of their book:

> *Historically, a course on data structures has been a mainstay of most computer science departments. However, over the last 15 years the focus of this course has broadened considerably. The topic of data structures now has been subsumed under the broader topic of* Abstract Data Type (ADTs) – *the study of classes of objects whose logical behavior is defined by the set of values and set of operations.*
>
> *The term* abstract data type *describes a comprehensive collection of data values and operations; the term* data structures *refers to the study of data and how to represent data objects within a program [...]. The shift in emphasis is representative of the move towards more abstraction in computer science education*. (p. v. The underline emphasis is added)

Typical data structures that are introduced in an introductory data structure course are: arrays, linked-lists, stacks, queues and trees. In most textbooks data structures are first introduced intuitively, based on real life examples with which students are familiar. For example, a queue is associated with a line of waiting people; a stack is associated with a stack of cafeteria trays, and so on. The operations themselves may also be explained in terms of these examples taken from students' daily experience. However, abstract definitions which are not based on particular examples, should be introduced to the students at some stage.

As in the case of abstract algebra, students are not familiar with this definition style. However, in contrast to the case of abstract algebra, in the case of data structures students are usually not asked to work solely with the abstract definitions. As there are tasks that require concrete treatment of data structures, for example, in the implementation of a specific data structure in a particular programming language, students may use this concrete instance of

the concept as a mental object to lean on when thinking about the abstract data structure. Still, introducing students to the abstract aspect of data structures may equip them with a cognitive tool with which to work in software development.

Differential Equations

A differential equation is an equation in which the derivatives of one or more functions of one or more independent variables appear (Collatz, 1986, p. 1). Thus, the objects with which the field of differential equations deals are functions, derivatives, and solutions of differential equations. At the stage where students meet the concept differential equation, they are only familiar with equations of numerical variables. Thus, when students meet the definition of differential equation for the first time, they have to generalize the concept of equation that they already know, and to also include equations whose variables are functions and derivatives of functions, into it.

Introductory differential equation courses usually present the mathematical reasoning of the methods described in the course for solving differential equations, as well as the properties of the solutions of differential equations. Specifically, existence and uniqueness of solutions are two important properties of solutions that are conveyed in introductory differential equations courses. The main activity students work on in an introductory differential equation course is equation solving, and it is towards this end that different solving techniques are presented for solving different kinds of equations. Such course orientation together with the fact that ''[a] course in elementary differential equations is an excellent vehicle through which to convey to the student a feeling for the interrelation between pure mathematics, on the one hand, and the physical sciences or engineering, on the other'' (Boyce & DiPrima, p. 7), may enhance students' feeling that the computational aspect of the course is the important one.

So far four topics have been described. The four fields address compound concepts from the very beginning of the respective introductory course: in abstract algebra these are groups; in data structures these are abstract data types; in computability these are languages and sets of languages; and in differential equation these are equations whose variables are functions and derivatives. Furthermore, properties of these concepts as well as relations between the concepts are examined. As is illustrated in Coping With Abstraction section, some students are unable to mentally construct such complex objects as these. As a result, in order to cope with a given problem-

solving situation, students make these concepts more mentally accessible by reducing the level of abstraction.

COPING WITH ABSTRACTION: REDUCING ABSTRACTION IN THE CONTEXT OF MATHEMATICS AND IN THE CONTEXT OF COMPUTER SCIENCE

This section addresses problem-solving situations in which students strive to give the concepts involved some meaning. As it turns out, the meaning that students give to some of the concepts can sometimes be interpreted as being on a lower level of abstraction than that on which concepts are introduced in class. In other words, the level of abstraction is reduced.

In this section three interpretations for abstraction are discussed, and the query ''what it means to reduce the level of abstraction'' is examined and illustrated for each interpretation with respect to each of the four fields – abstract algebra, computability, data structures and differential equations. The data is taken mainly from the following sources:[2]

Abstract Algebra – Hazzan (1995, 1999)

The theme of reducing abstraction was first introduced in this Ph.D. research. The data analysis was based mainly on interviews with nine undergraduate students who learned their first abstract algebra course. Each student was interviewed five times during the semester period. In addition, data was gathered from written research questionnaires, and regular classroom tests and homework assignments. The theoretical framework of reducing abstraction was developed in the spirit of 'grounded theory' (Glaser & Strauss, 1967), that is, the *data* have guided a successive development of the theoretical organization. Glaser and Strauss explain that ''[g]enerating a theory from data means that most hypotheses and concepts not only come from the data, but are systematically worked out in relation to the data during the course of the research.'' (p. 6). This attitude, which intertwines the development of a theory with the research process itself, is especially suitable in cases in which a new field is investigated. Hazzan (1995) was one of the first research attempts to examine comprehensively students' learning of abstract algebra.

[2]In addition to these resources, data from other researches are presented as well. These additional references are mentioned when they are quoted.

Computability – Hazzan (in press, 2003)
The work described in this article illustrates an application of the theme of reducing abstraction to other fields different from abstract algebra, for which the theme of reducing abstraction was originally developed. Data for this research was gathered from 115 students' answers to questions on a midterm exam in an Introduction to Computability Course. Complementary supportive data was collected from a make-up exam that was taken at the end of the semester by 69 students who either had not taken the midterm exam or wanted to improve their midterm exam grades. The data was analyzed both from a quantitative perspective and from a qualitative perspective through the lens of reducing abstraction. The suitability of the theme of reducing abstraction for analyzing students' understanding of concepts in computability was acquired on the basis of the research in advanced mathematical thinking and on the basis of the analysis of the nature of concepts learnt in computability theory.

Abstract Data Types – Aharoni (1999, 2000)
These publications describe Aharoni's Ph.D. research which was conducted as a qualitative case study. The main data collection tool was the semi-open, ethnographic, observational interview. A sequence of interviews was conducted with nine undergraduate students who learned in the data structures course. Observations were also gathered from observing classes on data structures and from occasional talks with people involved in various levels of computer science – from students to seniors. Since data structures are mathematical concepts in nature, and since no research on the learning of data structures was found at the time that that research was conducted, Aharoni used studies on mathematical thinking both for the research background and for the data analysis. Specifically, data was analyzed according to theories such as the actions-process-object theory (cf. Abstraction Level as Reflection of the Process-object Duality section) and that of the fragility of knowledge (Brousseau & Otte, 1991; diSessa, 1988; Smith, diSessa, & Rochelle, 1993).

Differential Equations – Raychaudhuri (2001)
This research is Raychaudhuri's Ph.D. thesis. The data were gathered from five semi-structured clinical interviews of six undergraduate students learning in an introductory course on elementary differential equations. Supplemental data is provided by classroom observations and notes, student quizzes and exams, and a questionnaire completed by the students after the completion of

the course. One of the theoretical contributions of this research is the adaptation, refinement and extension of the framework of reducing abstraction for explaining the patterns of construction that emerged from the data.

Abstraction Level as the Quality of the Relationships Between the Object of Thought and the Thinking Person

This interpretation for abstraction stems from Wilensky's (1991) assertion that whether something is abstract or concrete (or anything on the continuum between those two poles), is not an inherent property of the thing, ''but rather a *property of a person's relationship to an object*.'' (p. 198). In other words, for each concept and for each person we may observe a different level of abstraction which reflects previous experiential connection between the two. The closer a person is to an object and the more connections he/she has formed to it, the more concrete (and the less abstract) he/she feels towards it. Furthermore, what is abstract for someone at one time may be quite concrete for that same person at a later time. Based on this perspective, some of students' mental processes can be attributed to their tendency to make an unfamiliar idea more familiar or, in other words, to make the abstract more concrete.

This point of view is consistent with the perspective of Hershkowitz, Schwarz, and Dreyfus (2001) that emphasizes the learner's role in the abstraction processes. They claim that ''abstraction depends on the personal history of the solver'' (p. 197). Specifically, based on Davidov's theory (1972/1990) they claim that ''when a new structure is constructed, it already exists in a rudimentary form, and it develops through other structures that the learner has already constructed'' (p. 219).

Example from Abstract Algebra (Hazzan, 1999)

Groups of numbers (such as Z in relation to addition or $R - \{0\}$ in relation to number multiplication) are only some of the examples students come across in standard abstract algebra courses. Since each of these groups possesses additional properties such as commutativity and ordering, none is a typical example of a general group. Generic examples of groups, which students come across in abstract algebra courses, are permutation groups and groups of symmetries of regular polygons. Data analysis indicates that students often lean on their previous rich mathematical experience with numbers and treat groups as if they were made only of numbers and of operations defined on numbers.

In the following excerpt, for example, Tamar tries to determine whether Z_3 (i.e., the set {0, 1, 2} with addition mod 3) is a group. She takes for granted that 'the inverse' of 2 is $\frac{1}{2}$ (as with rational numbers in relation to number multiplication), and tries to locate it in Z_3. Since she does not find $\frac{1}{2}$ in Z_3 she concludes that Z_3 is not a group:

> Tamar: [Z_3] is not a group. Again I will not have the inverse. I will not have one half. I mean, 1 over . . . I mean, if I define this [Z_3] with multiplication, I will not have the inverse for each element. [. . .] It will not be in the set. [. . .] I'm trying to follow the definition. [. . .] What I mean is that I know that I have the identity, 1. What I have to check is if I have the inverse of each . . . I mean, I have to see whether I have the inverse of 2 and I know that the inverse of 2 is $\frac{1}{2}$. [. . .], and on top of that mod 3 [. . .] it is not included. I mean, I don't have the inverse of . . . My inverse is not included in the set. Then it's not a group. I do not have the inverse for each element.

In this excerpt Tamar automatically considers the group operation as number multiplication and thus replaces the less familiar operation (addition mod 3) with a more familiar one. This may be a result of the use of the term 'multiplication' for a general binary operation in the group definition, together with the need to mentally hang on to some familiar entity. However, it is worthwhile to notice that Tamar uses the theoretic-terms (identity, inverse) correctly, and since a single axiom failure denies a group, her explanation (had to be based on the group operation she considered) makes sense.

Example from Computability (Hazzan, in press, 2003)
In this example students make the unfamiliar familiar by mentally hanging on the familiar mathematical concepts of function. Specifically, it is illustrated how students rely on the concept of function that appears in the definition of the following language L:

$$L = \{(\langle M_1\rangle, \langle M_2\rangle) : f_{M1}(M_2) = f_{M2}(M_1) = 01001\},$$

where $f_M(x)$ is the function that M calculates with x as input.

In the mid-term exam on which this example is based, students were asked to determine for each of four languages whether it is recursive or recursively enumerable. Let us compare the above L with the language L_1=$\{(\langle M\rangle, x) : \exists M', x \notin L(M) \cap L(M')\}$. As the definition of L_1 includes quantifiers and set

Table 1. Grades of Students Whose Solution of L (Q4 in the Table) was Better than Their Solution of L_1 (Q1 in the Table).

Student #	Q1	Q2	Q3	Q4	Total
1	0	5	2.5	12	19.5
2	0	5	10	24	39
3	5	0	15	15	35
4	0	0	9	10	19
5	0	2.5	4	12.5	19
6	5	5	16	17	43
7	0	2.5	2.5	13	18
8	0	15	16	19	50
9	2	0	14.5	12.5	29
10	0	5	25	20	50
11	0	15	24	25	64
12	0	19	20	20	59

notations it seems that the definition of L_1 is more complex than the definition of L. In contrast, L's description uses functions, objects with which students have been familiar for many years. Surprisingly, data indicate that student solutions with respect to L_1 were better than their solution with respect to L. However, there were 12 students who worked on L relatively well but did not solve L_1.

A closer look at these 12 exams supports the assumption that the functions appearing in the definition of L were, in fact, familiar objects on which students could rely. As it turns out, these 12 students were weak students, whose average grade of the entire exam was 37.4 (out of 100). However, the familiar object – function – on which L's definition is based, helped them to do well at least on this part of the exam. Indeed, as is presented in Table 1, in the exams of these students the grade of Q4 that addresses L is the higher one, and in most of the cases students received most of their exam points from their work on L. From the perspective of reducing abstraction it is suggested that by mentally hanging on to a familiar mathematical concept (in our case – a function), students were able to give L some meaning and to solve relatively well the question that addressed it.

Example from Data Structures (Peled, 2001)

Peled (2001) researched students' conception of arrays when the students were in the process of learning programming in Visual Basic. According to one of her findings students believe that an array should contain at least two

elements. Thus, students do not conceive a structure which fulfills all the properties of array but includes only one element to be an array. The following excerpt illustrates this observation:

Researcher: What is the simplest kind of array that you know?
[. . .]
Researcher: Is an array with one element an array?
Limor: No.
Researcher: How many places should it have?
Limor: At least two. No? I'm hesitating.

Later on, Limor worked with an array with two elements. After taking away one element, an array with one element remained. She was then asked again:

Researcher: What is the thing you just took an element out of?
Limor: It is also an array. [Wonders] It is an array. And it has only one element. So, there is an array with only one element. But why? Why do we need it? What purpose does it serve?

This observation is similar to mathematical cases in which students do not conceive either a set with one element (a singleton) as a set or the empty set as a set. Zazkis and Gunn (1977) add to this conception saying that students confuse the notions of set-element with a one-element subset.

It is suggested that students adopt this belief from real life situations. Indeed, leaning on real life situations, there is no need to group one item in a container (the analogue to a set or to an array). One item can be carried simply as is. It is observed once again that when students meet objects which are abstract for them, students rely on their previous experience, possess the unknown object familiar properties, and thus, reduce the level of abstraction.

Example from Differential Equations (Raychaudhuri, 2001)

This example illustrates, in the context of differential equations, students' tendency to generalize their conclusions by relying on numbers – mathematical entities they feel close to and familiar with. In what follows Mika compares a solution of a differential equation with that of an equation:

Mika: A solution of an equation is either a number . . . or couple of numbers or infinite numbers that will make the equation true . . . whereas in differential equation you are solving . . . you can get . . . whole new function to be a solution to it . . . I

> think . . . so wouldn't be so much as just a number . . . guess it could be but as much as it could be a function . . . would be a solution. (p. 174)

Raychaudhuri explained that "[f]irst, to Mika the solution to a differential equation is defiantly more than a number, in fact, it is a "whole new function", and secondly, that along with being a function, she thinks it also has the potential to be a number ("guess it could be")". It is reasonable to assume that it is easier for Mika to mentally connect a number to her mental image of the concept of solution, as, in Mika's previous mathematical experience, the object of number was *the only* object that could form a solution.

Closing Remark: Abstraction Level as the Quality of the Relationships Between the Object of Thought and the Thinking Person

In this section the theme of reducing abstraction is expressed by students' mental processes which can be attributed to students' tendency to make an unfamiliar idea more familiar. What these examples have in common is the students' tendency to employ familiar objects when they (the students) are thrown into a problem-solving situation in which they are fumbling in the dark without any mental object to hang onto. Instead of revealing the meaning of the new situation, students *unconsciously* find ways to come out of the dark by relying on a familiar object. From the perspective of the interpretation of abstraction discussed in this section, abstraction level is reduced in all these examples.

Abstraction Level as Reflection of the Process-Object Duality

In this section the idea of reducing abstraction is reviewed based on the process-object duality suggested by theories of concept development in mathematics education (Beth & Piaget, 1966; Dubinsky, 1991; Sfard, 1991, 1992; Thompson, 1985). Some of these theories, such as the APOS (action, process, object and scheme) theory, suggest a more elaborated hierarchy (cf. Dubinsky, 1991). However, the discussion in this paper focuses on the process-object duality.

Theories that discuss this duality distinguish between process conception and object conception of mathematical notions. Dubinsky (1991) captures the passage from the first conception to the second one as a "conversion of a (dynamic) process into a (static) object". *Process* conception implies that one

regards a mathematical concept ''as a potential rather than an actual entity, which comes into existence upon request in a sequence of actions.'' (Sfard, 1991, p. 4). When one conceives of a mathematical notion as an *object*, this notion is captured as one ''solid'' entity. Thus, it is possible to examine it from various points of view, to analyze its properties and its relationships to other mathematical notions and to apply operations on it.

The mental process that leads from process conception to object conception is one mechanism Piaget named *reflective abstraction*. Dubinsky's work (1991) tells us about the importance of this mechanism for mathematical thinking: ''Piaget considered that it is reflective abstraction in its most advanced form that leads to the kind of mathematical thinking by which form or process is separated from content and that processes themselves are converted, in the mind of the mathematician, to objects of content.'' (p. 98). According to these theories, when a mathematical concept is learned, its conception as a process precedes – and is less abstract than – its conception as an object (Sfard, 1991, p. 10). Thus, process conception of a mathematical concept can be interpreted as on a lower level of abstraction than its conception as an object.

Students' conception of concepts as processes is illustrated in this section mainly by students' tendencies to work with canonical procedures. A *canonical* procedure is a procedure that is more or less automatically triggered by a given problem. This can happen either because the procedure is naturally suggested by the nature of the problem, or because prior training has firmly linked this kind of problem with this procedure. The availability of a canonical procedure enables students to obtain a solution without worrying too much about the mathematical properties of the concepts involved. It seems that this technical work gives students the assurance of following a well-known, step-by-step procedure, where each step has a clear outcome. In contrast, relying on abstract reasoning, for example by exploring properties of concepts or by relying on theorems, may be shaky mental approach for the students. Using the process-object duality terminology we may say that solving a problem by relying on a canonical procedure is an expression of process conception of the concepts under discussion; solving a problem by analyzing the essence and properties of concepts is an expression of object conception of the concepts under discussion.

The following examples illustrate, in a variety of problem-solving situations taken from the four fields, how process conception of concepts is expressed by a solution that is based on carrying out a canonical procedure. This approach allows students to bypass analysis of concept properties,

that could have been carried out had the concepts been conceived as objects.

Example from Abstract Algebra (Hazzan, 1999)
This example illustrates students' tendency to work with canonical procedures rather than relying on theorems. As theorems usually state properties of concepts or relationships among concepts, it is argued that the use of theorems in problem-solving situations requires an object conception of the concepts appearing in the problem. Thus, when students base their solution on a canonical procedure instead of relying on theorems, it is reasonable to assume that process conception of the involved concepts is expressed.

In the following excerpt Guy presents a clear analysis of the structure of the cosets in a group, but then ''forgets'' to use that knowledge. In the solution of another problem, he chooses the alternative of a messy coset calculation. Here is his description:

> Guy: OK, it is known that equivalent classes divide the set into disjoint sets, the whole set. And now, if a coset is like an equivalent class then the cosets divide the . . . divide the group into disjoint sets. We know that in each coset the number of elements is equal.

In spite of this clear description, when asked to calculate the cosets of the subgroup $\{1, 2, 4\}$ in $Z_7 - \{0\}$ with multiplication mod 7, Guy first calculated all six cosets (one for each element of the group). It was only when he noticed that there were just two different cosets, that he began to look for the reason.[3] It is suggested that despite Guy's clear description of the cosets as equivalent classes with the same number of elements, Guy still conceives the concept of coset as a process. As Guy's conception of cosets is process oriented, his first tendency was to start calculating the cosets by using a well-known procedure for this kind of task. It is reasonable to assume that the property-oriented knowledge (i.e., the more abstract knowledge) would have popped up first had Guy's perception of coset been object oriented.

Example from Computability (Hazzan, in press, 2003)
This examples compare two methods for proving that $L = \{\langle M\rangle : \exists M', \langle M'\rangle \in L(M) \wedge L(M') \neq \phi\}$ is not in R: Using Rice's theorem and construction

[3]Since there are only two distinct cosets, there is no need to do *any* calculation whatsoever: one coset must be the subgroup $\{1, 2, 4\}$ itself, and the other one must be its complement $\{3, 5, 6\}$.

of a reduction from one of the not recursive languages. Here is a short explanation of the two strategies. Further details are presented in the Appendix.

Using Rice's theorem: Rice's theorem provides criteria for determining for a given language L of a certain form whether $L \notin R$ and whether $L \notin \text{RE}$. Using Rice's theorem requires that one's argument be based on identifying a non-trivial property of languages in RE.

Defining a reduction between two languages L' and L'', for one of which it is known that it is or is not in R or in RE: In general, if a reduction from L' to L'' exists, it indicates that L'' is at least as difficult as L'. Thus, when we define a reduction from a language L', $L' \notin R$ (or $\notin \text{RE}$) to a language L'', it is proved that $L'' \notin R$ (or $\notin \text{RE}$). When a reduction is defined from L' to L'', for some $L'' \in R$ (or $\in \text{RE}$), it is proved that $L' \in R$ (or $\in \text{RE}$). A construction of a reduction consists of two steps. First, finding the reduction, and second, proving that the reduction satisfies three conditions.

A comparison of the details involved in using Rice's theorem for determining whether or not the above language in R with the details one has to deal with if one builds a reduction for this purpose, indicates that relying on Rice's theorem is a simpler and shorter approach. However, the data indicate that students prefer building a reduction over using Rice's theorem (see Table 2). This observation may be explained by the fact that it is simpler to interpret reduction-building[4] as a process, than it is to rely on a theorem,

Table 2. The Number of Students Who Used Rice's Theorem and the Number of Students Who Built a Reduction in Showing that a Given Language is Not Recursive.

Showing that $L = \{\langle M \rangle : \exists M', \langle M' \rangle \in L(M) \wedge L(M') \neq \phi\} \notin R$	
No. of students who used Rice's theorem: 36 students.	
No. of students who built a reduction: 72 students. The students used different languages as the source language of the reduction. Here are the languages that were used (whether correctly or incorrectly) together with the number of students who used them	
HP	45
$L_{(\phi)'}$ or $(L_\phi)'$	12 (note: the symbol′ represents set complement)
L_{d}	12
L_{u}	3

[4]There is a distinction between the concept 'reduction' and the concept 'reduction-building'. Consequently, process conception and object conception for each of these two concepts are expressed differently. The discussion here focuses on students' conception of reduction-building. Process conception of reduction-building is expressed by constructing a reduction, by following some procedure mechanically, without necessarily understanding each step.

which, as in the case of the abstract algebra example, requires object conception of the concepts appearing in the theorem.

The above explanation suggests that students tend to define reductions rather than using Rice's theorem as a result of process conception of reduction-building. This explanation can be refined if we observe the dominance of HP as a source of the reduction. A superficial explanation may bind this dominance with the fact that students practiced this kind of reduction in class and in the tutorials. However, it is further suggested that it is relatively easy to conceive 'reduction-building from HP' as a process because a canonical procedure for this purpose exists. This idea was expressed in an occasional conversation with one of the students. The student explained that defining a reduction from HP is the easiest way of solving questions of this kind, because there is a pattern that one can apply when a reduction from HP is constructed. He said: "*When I construct a reduction from HP I know what to do in each case, both when it halts and when it does not halt. [. . .] When I learn for the exam [and solve problems from previous years] instead of complicating the solution, showing that a property is a non-trivial property of languages in RE [and using Rice's theorem], I always construct a reduction from HP*". In other words, the process of constructing a reduction from HP becomes a procedure that can be mimicked without analyzing the nature and properties of the concepts with which one works.

Example from Data Structures (Aharoni, 2000)

This example illustrates student perception of concepts as processes in the context of data structures by students' tendency to express what Aharoni calls *programming oriented thinking*. Aharoni discusses students' responses to the question: "What is an array?". According to Aharoni, from the point of view of the data structures domain, a general answer to this question would be an answer that refers to an abstract array, as the following description does: "An array is a collection of ordered pairs (index-set, value), where all the index-sets are distinct, together with the operations INSERT (inserting a new pair into the array) and GET (returning the value of a specified index)." As it turns out, when the question "What is an array?" was presented to students, the majority of answers expressed the orientation reflected in the following one:

Roy: An array is a continuous area in the [computer's] memory, which holds elements of the same type. We can access each element by specifying its index, which is a whole number.

Roy talks about the concept of array as being implemented in some computer memory. Aharoni suggests that such a definition reflects *programming oriented thinking*, by which a learner has to think of the data structure in terms of being implemented within some computer. In contrast, Aharoni argues that a general answer, like the one mentioned above, reflects *programming-free thinking* and that this mode of thinking may exist only if the concept of the data structure at hand is conceived as an object.

Example from Differential Equations (Raychaudhuri, 2001)
This example illustrates how students give up the use of a mathematical definition that describes a mathematical entity by its properties, and utilize a canonical procedure which is described by them as being an easier tool. Specifically, this example shows how students calculate Wronskian determinant instead of using the definition of linear independence.

As is described by Raychaudhuri (2001), most students are familiar with the definition of linear independency of two functions. However, once a computation for checking the linear independency of two functions is introduced, students opt to use it instead of the definition. In other words, they prefer the use of a process over the use of a definition that captures the properties of the concept.

More specifically, *the definition* of linear dependency states that two functions y_1 and y_2 are linearly dependent on an interval if there exist two constants c_1 and c_2, not both zero, such that $c_1y_1 + c_2y_2 = 0$. *The process* that allows for checking the linear dependency of two functions on an interval is based on the calculation of a determinant. It says that two functions y_1 and y_2 are linearly dependent on an interval if the Wronskian:

$$W(y_1, y_2)(x) = \begin{vmatrix} y_1(x) & y_2(x) \\ y_1'(x) & y_2'(x) \end{vmatrix}$$

is non-zero for some x in that interval.

The following excerpt illustrates students' confidence in canonical procedures while losing faith in definitions, even when simple objects are involved. Omar offers x and $2x$ as a pair of linearly dependent functions, but then he wants to check his answer.

Omar:	[. . .] I'm going to look at that for a sec . . . [computes Wronskian] yeah . . . x and $2x$ . . . they are linearly dependent.
Researcher:	Why is that?
Omar:	Well, you take the Wronskian and they are equal to 0 . . . so . . .

Researcher: So when you constructed it [x and $2x$], did you take Wronskian [of the functions] in your mind?

Omar: Yeah . . . first . . . oh! . . . no, I was pretty sure that these are dependent . . . but just to make sure . . . I did the Wronskian. [emphasis added]

Researcher: So you did the Wronskian to double check . . . that means you have another way of knowing if they are linearly dependent or not . . .

Omar: Right . . .

Researcher: Which is . . . ?

Omar: Oh . . . I just thought of two numbers [. . .] . . . $c_1y_1 + c_2y_2 =$ like whatever [. . .] if I change c_1 and c_2 around . . . that could come up with a 0.

Raychaudhuri (2001) claims that "[e]ven though they [students] could produce two linear independent functions using the definition, they proceeded to compute the Wronskian anyway to double check their results." (p. 191).

Closing Remark: Abstraction Level as Reflection of the Process-object Duality

In this section the theme of reducing abstraction is expressed by students' conception of abstract concepts as processes. In none of the cases did the students express object conception of the concept under examination. Object conception could have been expressed by examining relationships between two algebraic structures (a coset and a group), by using a theorem (Rice's theorem), by reflecting programming-free thinking (with respect to the data structure array), or by using a definition (of linear dependency of two functions). Rather, in all cases (except for the data structure example) process conception is expressed by using a well-known canonical procedure for solving the given problem. In all these manifestations, using canonical procedures seemed to be the most concrete path out of all those other alternatives mentioned above. From the perspective of the interpretation of abstraction discussed in this section, abstraction level is reduced in all these examples.

Abstraction Level as the Degree of Complexity of the Concept of Thought

This section examines abstraction by the degree of complexity of the concepts of thought. It does not imply, of course, that it should be more difficult to think

in terms of compound objects. The working assumption here is that the more compound an entity, the more abstract it is. In this respect, this section focuses on how students reduce abstraction level by replacing a set with one of its elements, thus, working with a less compound object. As it turns out, this is a handy tool when one is required to deal with compound objects that have not yet been fully constructed in one's mind.

Example from Abstract Algebra (Hazzan, 1999)
This example describes a student who faced a problem concerning a set of groups, and turned to consider only one group from the whole set (cf. also Rumelhart, 1989, p. 301: reasoning by example). In other words, he replaced a set with one of its elements. Considering a specific case may be a positive and helpful heuristic, as recommended by Polya (1973): "Specialization is passing from the consideration of a given set of objects to that of a smaller set, or of just one object, contained in the given set. Specialization is often useful in the solution of problems." (p. 190). The role of such specialization is to guide towards a general solution. However, there are cases in which students do use this heuristic, presenting an answer based on an analysis of a specific case, and do *not* return to the general case. Sometimes they do not go back to the general case simply because they are unable to do so – the mental structures needed to deal with the general case have not yet been constructed.

In the following excerpt Ron addresses the following question: "Does a group of even order always have a subgroup of order 2?". In the following excerpt Ron describes how he "just misses the point" and thus turns to work with a specific group. Apparently, he encounters difficulties in thinking about the family of groups of even order or difficulties in thinking about a generic group of even order.

> Ron: A group of even order . . . does it always have a subgroup of order 2? [. . .] [pause] Why yes or why no? I just feel that I miss the point. [. . .] A group of even order can also be a group . . . let's say of order 6 for example. A group of even order? [. . .] Yes. If the order is 6 so it can have subgroups [of order] either 2 or 3, of order 2 or order 3.

Example from Computability (Hazzan, in press, 2003)
This example looks at the following language presented in a midterm exam.

Let S be a non-trivial property of languages in RE.

$$L = \{(\langle M_1 \rangle, \langle M_2 \rangle) : L(M_1) \in S \wedge L(M_2) \notin S\}$$

In what follows it is illustrated how students reduced the level of abstraction of the object they had to mentally deal with. This was done with respect to the non-trivial property *S*. Thinking about a property of languages requires mentally distinguishing between a language and a property of languages (the latter is a set of languages). In our case, there is a need to conceive that *S* is *any* non-trivial property. The mental processes and constructions needed for understanding the two objects (*specific* non-trivial property and *any* non-trivial property) are on different levels of abstraction since a *specific* non-trivial property is, in itself, a *set* of *languages.* As *any* property is more compound, hence a more abstract concept than a specific property, many students could not mentally access it, and (unconsciously) approached the solution by choosing a specific property.

Table 3 lists the specific properties that students chose when they solve the question, together with the grades the students got in the entire exam. The reason for listing the students' grades is to illustrate that most of the students (except 1) who chose a specific property were not necessarily weak students. Still, they encountered difficulties in mentally accessing the object of *any* non-trivial property. From the perspective of reducing abstraction we can see that instead of dealing with a set of properties (the set of non-trivial properties), students reduced the level of abstraction and approached the solution by mentally referring to a specific non-trivial property.

Table 3. Specific Properties Students Chose in Addressing *L*, the Number of Students Who Chose Each of the Properties and the Students' Grades in the Exam.

Chosen property	Number of students and their grade in the exam
$S = \{\Sigma^*\}$ This S was also described as $S = \{L \in \text{RE} \mid L = \Sigma^*\}$ or $S = \{L \in \text{RE} \mid L = \Sigma^*\} = \{\Sigma^*\}$	4 students. Their grades: 84, 71, 85, 85
$S = \{\phi\}$ or $S = \{L \in \text{RE} \mid L = \phi\}$	4 students. Their grades: 86, 85.5, 73, 81
$S = \{L \in \text{RE} \mid \varepsilon \in L\}$	2 students. Their grades: 62.5, 82
$S = \{L \mid \mid L \mid = 3\}$ or $S = \{L \mid \mid L \mid = 0\}$	1 for each number of words in L. Their grades: 20 and 87.5
Other specific S	6 students

Example from Data Structures (Aharoni, 1999)
This example presents the phenomenon that Aharoni describes as thinking with prototypes. Aharoni (1999) observes that students think about abstract data structures by imaging a prototype for each of them. For example, with respect to the abstract data type array, Aharoni identifies a *linear* array to be students' prototype. Indeed, as the following excerpt illustrates, students reduce the complexity of the object of thought by referring to a linear array as the prototype for the concept of array (p. 131):

> Hadas: [tries to categorize data structures with which she is familiar] One-dimensional and two-dimensional [. . .] array, one-dimensional array. Table, set, stack, and queue, we talk about one-dimensional. [. . .] Trees, all kinds of trees, heaps, all are two-dimensional things. [. . .] If you add dimensions, it will be harder to program, more difficult to get, to see it with our eyes. [emphasis added]

With respect to the abstract data structure linked-list, Aharoni observes that students' prototype is a *linear* linked-list. Indeed, as the following excerpt illustrates, students reduce the complexity of the object of thought by referring to this kind of list as the prototype for linked-list.

> Hadas: A linked-list is . . . if you do not provide me with additional information . . . it is usually one-directional [one-dimensional list], because there are also two-directional [lists] that other operations are defined for them.

As can be observed, when Hadas speaks about linked-lists she talks about one-dimensional (''one-directional'' as she calls it) lists. Hadas knows about the existence of a linked-list with several dimensions, but the prototype – the linear, one-dimensional linked list – is the list to which she usually refers.

Example from Differential Equations (Raychaudhuri, 2001)
This example shows that students' conception of a solution of a differential equation is limited to the solutions obtained from differentiating the equation. In other words, Raychaudhuri says that ''[s]tudents think that the solution obtained upon integrating a differential equation provides all possible solutions to the differential equation. They demonstrate this image by identifying the statement 'A general solution to a differential equation always contains all the solutions to it' as true [. . .], with general solution defined by

the students as the solution obtained upon integration.'' The following quote shows how Peter believes that it is impossible to have a solution without that solution being part of a general solution:

Researcher: Is this statement true or false? 'A differential equation can have a particular solution even in the absence of a general solution'.

Peter: I think that's false . . . because when you first get the solution of the differential equation you get the general solution. That means, you don't have the general solution, you don't have a solution.

This example shows that students conceive the notion of 'a solution to a differential equation' in a more limited manner than that of its meaning. Thus they reduce the complexity of the concept of set of solutions. In terms of the interpretation of abstraction discussed in this subsection, the level of abstraction is reduced.

Closing Remark: Abstraction Level as the Degree of Complexity of the Concept of Thought

In this section, the theme of reducing abstraction is expressed by students' tendency to reduce the complexity of the object of thought in different contexts. In all of these cases, students unconsciously mentally 'replace' a set of objects with one element of the set or with a subset of the set. From the perspective of abstraction discussed in this section, the level of abstraction is reduced.

SUMMARY

The article illustrates how the theme of reducing abstraction can be applied to analyzing students' understanding of topics in mathematics and in computer science. In a broader perspective, it is suggested that as computer science education is a relatively young discipline, it may benefit from borrowing learning theories already developed in mathematics education over the last decades. Specifically, it is suggested to explore the understanding of other computer science topics through the lens of reducing abstraction. Such analysis may be applied mainly to topics that deal with complex concepts from the very beginning of the respective introductory courses.

Such as analysis may be useful for instructors who teach mathematics and computer science courses. The reason is that good educators continually seek that thin line between abstract (for the students) and concrete (for the students) in order to avoid working too abstractly or too concretely, respectively. Being aware of students' tendency to reduce the level of abstraction, may improve student-teacher communication.

ACKNOWLEDGEMENT

I would like to express my gratitude to Rina Zazkis for her useful and thoughtful remarks on an earlier draft of this article.

REFERENCES

Aharoni, D. (1999). *Undergraduate students' perception of data structures*. Ph.D. Thesis, Technion – Israel Institute of Technology, Haifa, Israel.

Aharoni, D. (2000). Cogito, Ergo Sum! Cognitive processes of students dealing with data structures. In S. Haller (Ed.), *Proceedings of the Thirty-first SIGCSE Technical Symposium on Computer Science Education (SIGCSE 2000)*. Austin, TX: The Association for Computing Machinery.

Almstrum, V.L., Ginat, D., Hazzan, H., & Morley, T. (2002). Import and export to/from computing science education: The case of mathematics education research. *The Seventh Annual Conference on Innovation and Technology in Computer Science Education* (*ITiCSE 2002*), Aarhus, Denmark.

Beth, E.W., & Piaget, J. (1966). *Mathematical epistemology and psychology*. D. Reidel, Dordrecht, The Netherlands.

Boyce, W.E., & DiPrima, R.D. (1965). *Elementary differential equations and boundary value problems*. New York: Wiley.

Brousseau, G., & Otte, M. (1991). The fragility of knowledge. In A.J. Bishop, J. van Dormolen, & S. Mellin-Olsen (Eds.), *Mathematical knowledge: Its growth through teaching* (pp. 13–36). The Netherlands: Kluwer Academic Publishers.

Collatz, L. (1986). *Differential equations*. Great Britan: Wiley.

Davydov, V.V. (1990). *Soviet studies in mathematics education:* Vol. 2. *Types of generalization in instruction: Logical and psychological problems in the structuring of school curricula* (J. Kilpatrick, Ed., & J. Teller, Trans.). Reston, VA: National Council of Teachers of Mathematics (Original work published in 1972).

diSessa, A.A. (1988). Knowledge in pieces. In G. Forman & P. Pufall (Eds.). *Constructivism in the computer age*. NJ: Lawrence Erlbaum Associates, Inc.

Dubinsky, E. (1991). Reflective abstraction in advanced mathematical thinking. In D. Tall (Ed.), *Advanced Mathematical Thinking* (pp. 95–123). Dordrecht: Kluwer Academic Press.

Dubinsky, E., Dautermann, J., Leron, U., & Zazkis, R. (1994). On learning fundamental concepts of group theory. *Educational Studies in Mathematics*, *27*, 267–305.

Dubinsky, E., & Leron, U. (1994). *Learning abstract algebra with Isetl*. Berlin: Springer.
Gallian, J.A. (1990). *Contemporary abstract algebra* (2nd ed.). Lexington, MA: D.C. Heath and Company.
Glaser, B., & Strauss, A.L. (1967). *The discovery of grounded theory: Strategies for qualitative research*. Chicago: Aldine.
Hazzan, O. (1995). *Undergraduate students' understanding of concepts in abstract algebra*. D.Sc. Thesis, Technion – Israel Institute of Technology, Haifa, Israel.
Hazzan, O. (1999). Reducing abstraction level when learning abstract algebra concepts. *Educational Studies in Mathematics*, *40*(1), 71–90.
Hazzan, O. (in press, 2003). Reducing abstraction when learning computability theory. *Journal of Computers in Mathematics and Science Teaching*, *22*(2).
Hershkowitz, R., Schwarz, B.B., & Dreyfus, T. (2001). Abstraction in context: Epistemic actions. *Journal for Research in Mathematics Education*, *32*, 195–222.
Herstein, I.N. (1986). *Abstract algebra*. New York: Macmillan.
Hopcroft, J., Motwani, R., & Ullman, J.D. (2001). *Introduction to automata theory, languages, and computation* (2nd ed.). Addison-Wesley, Reading, MA.
Nell, D., & Teague, D. (2000). *C++ plus data structures*. Boston: Jones and Bartlett Publishers.
Peled, B. (2001). *Conception of the data structure array by "Handassaim" students*. M.Sc. Thesis, Technion – Israel Institute of Technology, Haifa, Israel.
Polya, G. (1973). *How to Solve it*? (2nd ed.). Princeton, NJ: Princeton University Press.
Raychauhuri, D. (2001). *The tension and the balance between one mathematical concept and student constructions of it: Solution to a differential equation*. Ph.D. Thesis, Simon Fraser University, Vancouver, Canada.
Rumelhart, D.E. (1989). Toward a microstructural account of human reasoning. In S. Vosniadou & A. Ortony (Eds.), *Similarity and analogical reasoning* (pp. 298–312). Cambridge: Cambridge University Press.
Sfard, A. (1991). On the dual nature of mathematical conceptions: Reflections on processes and objects as different sides of the same coin. *Educational Studies in Mathematics*, *22*, 1–36.
Sfard, A. (1992). Operational origins of mathematical objects and the quandary of reification – The case of function. In E. Dubinsky & G. Harel (Eds.), *The concept of function – aspects of epistemology and pedagogy.* MAA Notes.
Smith, J.P. III, diSessa, A.A., & Roschelle, J. (1993). Misconceptions reconceived: A constructivist analysis of knowledge in transition. *Journal of the Learning Sciences*, *3*(2), 115–163.
Thompson, P.W. (1985). Experience, problem solving, and learning mathematics: Considerations in developing mathematics curricula. In E.A. Silver (Ed.), *Teaching and learning mathematical problem solving: Multiple research perspective* (pp. 189–236). Hillsdale, NJ: Erlbaum.
Wilensky, U. (1991). Abstract meditations on the concrete and concrete implications for mathematical education. In I. Harel & S. Papert (Eds.), *Constructionism* (pp. 193–203). Norwood, NJ: Ablex Publishing Corporation.
Zazkis, R., & Gunn, C. (1997). Sets, subsets and the empty set: Students' constructions and mathematical conventions. *Journal of Computers in Mathematics and Science Teaching*, *16*(1), 133–169.

APPENDIX

The appendix presents concepts in each of the four fields discussed in the article. It presents only those facts required for understanding the examples presented in this article.

Basic Concepts of Abstract Algebra (Herstein, 1986)

Group	A nonempty set G is said to be a *group* if in G there is defined an operation $*$ such as: (a) $a, b \in G$ implies $a * b \in G$. (b) Given a, b, and $c \in G$, then $a * (b * c) = (a * b) * c$. (c) There exists a special element $e \in G$ such that $a * e = e * a = a$ for all $a \in G$ (e is called the *identity* or *unit element* of G). (d) For every $a \in G$ there exists an element $b \in G$ such that $a * b = b * a = e$ (We call it the *inverse* of a in G).
Subgroup	A nonempty subset, H, of a group G is called a *subgroup* of G if, relative to the product in G, H itself forms a group.
Coset	Let G be a group and H be a subgroup of G. For any $a \in G$, the set $aH = \{ah \mid h \in H\}$ is called the *left coset of H in G containing a*. Analogously, $Ha = \{ha \mid h \in H\}$ is called the *right coset of H in G containing a* (Gallian, 1990, p. 130).
Order of a group	A group G is said to be a *finite group* if it has a finite number of elements. The number of elements in G is called the *order* of G.
Order of a group element	If G is finite, then the *order* of a, written $o(a)$, is the *least positive integer m* such that $a^m = e$.

Basic Concepts of Computability

Let M be a Turing machine. Σ^* is the set of all strings over an alphabet Σ.

$$L(M) = \{w \mid w \in \Sigma^*\text{: when } M \text{ gets } w \text{ as input, } M \text{ halts in } q_{\text{accept}}\}$$

$R = \{L \mid L \text{ is decidable}\}$
$\text{RE} = \{L \mid \text{Exists } M, L(M) = L\}$

A property S of languages in RE: A set of languages in RE so that $S \subseteq \text{RE}$. A property is said to be *non-trivial* property if $\emptyset \neq S \neq \text{RE}$.

$L_s = \{\langle M\rangle : L(M) \in S\}$
($\langle M\rangle$ denotes the string that represents the Turing machine.)

Rice's theorem: Let S be a non-trivial property of languages in RE.

Then $L_s = \{\langle M\rangle : L(M) \in S\} \notin R$. If $\emptyset \in S$ then $L_s \notin \text{RE}$.

Reduction: A reduction f from L_1 to L_2 is a computable function, defined for all

$$x \in \Sigma^*, \text{ that satisfies } x \in L_1 \iff f(x) \in L_2.$$

The following list contains the main language classifications:
Note: The symbol $'$ represents a set complement.

$\text{HP} = \{(\langle M\rangle, x) : M \text{ halts on } x\} \in \text{RE}\backslash R$	$(\text{HP})' \notin \text{RE}$
$L_u = \{(\langle M\rangle, x) : M \text{ accepts } x\} \in \text{RE}\backslash R$	$(L_u)' \notin \text{RE}$
$L_D = \{\langle M\rangle : M \text{ accepts } \langle M\rangle\} \in \text{RE}\backslash R$	$(L_D)' \notin \text{RE}$
$L_\emptyset = \{\langle M\rangle : L(M) = \emptyset\} \notin \text{RE}$	$(L_\emptyset)' \in \text{RE}\backslash R$

Basic Concepts of Data Structures

The following definitions are quoted from the Dictionary of Algorithms and Data Structures, National Institute of Standards and Technology, http://www.nist.gov/dads/

Array	A set of items which are randomly accessible by index.
Stack	A collection of items in which only the most recently added item may be removed. The latest added item is at the top. Basic operations are push, pop, and top. Also known as ''last-in, first-out'' or LIFO.
Queue	A set of items in which only the earliest added item may be accessed. Basic operations are add (to the tail) or enqueue and delete (from the head). Delete returns the item removed. Also known as ''first-in, first-out'' or FIFO.
Tree	A data structure accessed beginning at the root node. Each node is either a leaf or an internal node. An internal node has one or more child nodes and is called the parent of its child nodes. All children of the same node are siblings. Contrary to a physical tree, the root is usually depicted at the top of the structure, and the leaves are depicted at the bottom.

Binary tree	A tree with at most two children for each node.
Heap	A tree where every node has a key more extreme (greater or less) than the key of its parent.

Basic Concepts of Differential Equations

Differential equation	A differential equation is an equation in which the derivatives of one or more functions of one or more independent variables appear (Collatz, 1986, p. 1).
General solution of a differential equation	The solution which contains all solutions of a differential equation. (Boyce & DiPrima, 1965, p. 18).
Particular solution of a differential equation	A solution of a differential equation satisfying prescribed initial conditions. (Boyce & DiPrima, 1965, p. 116).